LOAN MEME · SECURITY REVIEW

Loan Meme
Loan Protocol Vault.

A security review of Loan Protocol Vault.

CLIENT	PROJECT	AUDITORS	AUDIT WINDOW	FINAL REPORT
Loan Meme	Loan Protocol Vault	Roberto Reigada, Ferran Celades	September 24 – September 28, 2026	October 22, 2026

§ 00

Contents.

12 pages: overview, confidence ranking, scope, 3 findings, disclaimer, and about Guardian.

§ 01 **Overview** P. 03

§ 02 **Confidence Ranking** P. 04

§ 03 **Scope & Classifications** P. 05

Files in Scope P. 06

§ 04 **Findings** P. 07

Main Review · 3 findings P. 08

§ 05 **Disclaimer** P. 11

§ 06 **About Guardian** P. 12

§ 01

Overview.

Guardian reviewed Loan Protocol Vault across 2 review rounds. The final report includes 3 Informational findings.

AUDIT FIRM
Guardian

CLIENT
Loan Meme

PROJECT
Loan Protocol Vault

PREPARED BY
Roberto Reigada, Ferran Celades

AUDIT WINDOW
September 24 – September 28, 2026

FINAL REPORT
October 22, 2026

REPOSITORY
`https://github.com/creepdius/loan-smartcontracts`

METHODOLOGY
Smart contract · Manual · Main Review · Remediation Review

MAIN REVIEW COMMIT
`92dc8c9e47aa`

REMEDIATION REVIEW COMMIT
`841fa81bbc80`

1.1 Confidence Ranking

5 / 5

Very High Confidence

Guardian assigns a Confidence Ranking of 5 out of 5, combining finding severity, remediation outcome, and the reviewer's assessment of latent risk. The full scale is on the next page.

1	2	3	4	5
VERY LOW CONFIDENCE	LOW CONFIDENCE	MODERATE CONFIDENCE	HIGH CONFIDENCE	VERY HIGH CONFIDENCE

1.2 Findings at a Glance

SEVERITY	TOTAL	PENDING	ACK	PARTIAL	RESOLVED
Critical	**0**	0	0	0	0
High	**0**	0	0	0	0
Medium	**0**	0	0	0	0
Low	**0**	0	0	0	0
Informational	**3**	0	1	0	2
All severities	**3**	**0**	**1**	**0**	**2**

§ 02

Confidence Ranking.

Guardian assigns a project-level confidence score using the scale below. The ranking combines finding severity, remediation outcome, and the reviewer's assessment of latent risk.

CONFIDENCE RANKING	DEFINITION AND RECOMMENDATION	RISK PROFILE
5 Very High Confidence ASSIGNED	Codebase is mature, clean, and secure. No High or Critical vulnerabilities were found. Follows modern best practices with high test coverage and thoughtful design. Recommendation: code is highly secure at time of audit.	0 High/Critical findings and few Low/Medium severity findings.
4 High Confidence	Code is clean, well-structured, and adheres to best practices. Design patterns are sound and test coverage is strong. Recommendation: suitable for deployment after remediations; consider periodic review with changes.	0-1 High/Critical findings per engagement week and little to no Medium severity findings.
3 Moderate Confidence	Code is functional but shows gaps in defensive programming or test coverage. Recommendation: remediate the findings and re-review the changes before deployment.	Some High/Critical findings, with various Medium severity findings.
2 Low Confidence	Recurring issues suggest systemic gaps rather than isolated mistakes. Recommendation: remediate, then commission a follow-up review at a frozen commit.	Over 6 High and Critical findings across the engagement.
1 Very Low Confidence	The volume and severity of findings indicate the codebase is not ready for production. Recommendation: substantial rework, then a full re-review.	Over 12 High and Critical findings across the engagement.

§ 03

Scope & Classifications.

Guardian reviewed Loan Protocol Vault in `https://github.com/creepdius/loan-smartcontracts`. The scope includes 436 source lines across 1 file.

3.1 Severity Matrix

	IMP · HIGH	IMP · MED	IMP · LOW
LIK · HIGH	Critical	High	Medium
LIK · MED	High	Medium	Low
LIK · LOW	Medium	Low	Low

IMPACT · HIGH	Significant loss of assets, harm to a group of users, or core protocol functionality is disrupted.
IMPACT · MEDIUM	Small amount of funds can be lost, or ancillary functionality is affected. Users may experience reduced or delayed receipt.
IMPACT · LOW	Unexpected behavior in some functionality that is notable but does not meet the criteria for higher severity.
LIKELIHOOD · HIGH	Possible with reasonable assumptions that mimic on-chain conditions; cost is low compared to amount gained.
LIKELIHOOD · MEDIUM	Possible only in uncommon cases or requires a large amount of capital relative to disruption caused.
LIKELIHOOD · LOW	Unlikely to ever occur in production.

Files in Scope.

FILE	NSLOC	TOTAL
MAIN REVIEW		
contracts/Treasury.sol	218	300
REMEDIATION REVIEW		
contracts/Treasury.sol	218	300

§ 04

Findings.

Grouped by round. Visible IDs reset by severity within each round, so a finding is named the same way in the report as it was in the channel it was written in. Status is where each finding ended up; its own page says how.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
MAIN REVIEW					
I-01	Loan asset identity is inconsistent	Documentation	INFORMATIONAL	RESOLVED	p. 8
I-02	Runtime bytecode differs across instances	Documentation	INFORMATIONAL	ACKNOWLEDGED	p. 9
I-03	Cap invariant omits later cap reductions	Documentation	INFORMATIONAL	RESOLVED	p. 10

INFORMATIONAL **I-01** RESOLVED

Loan asset identity is inconsistent

CATEGORY	Documentation
LOCATION	`docs/AUDIT-SCOPE.md:18,35; docs/REQUIREMENTS.md:116`
REVIEW	Main Review

DESCRIPTION

The Treasury is intended to hold an ERC-20 loan asset whose address and caps in raw token units are supplied at deployment. `docs/AUDIT-SCOPE.md` calls that asset USDG in its overview but USDC in its trust model. `docs/REQUIREMENTS.md` then describes USDG as Robinhood's native currency, although the scope excludes native currency and `payout()` transfers through `IERC20.safeTransfer`. These labels leave the token to configure and fund unclear. If USDG is the intended loan asset, an operator following the USDC label can configure or fund the wrong token, leaving the intended payouts unavailable until the configuration and funding are corrected.

RECOMMENDATION

Identify the intended ERC-20 loan token consistently in the scope and requirements, including its target chain contract address and decimals. Correct the native currency entry.

RESOLUTION

Status: Resolved.

INFORMATIONAL I-02

ACKNOWLEDGED

Runtime bytecode differs across instances

CATEGORY	Documentation
LOCATION	docs/REQUIREMENTS.md:78; docs/AUDIT-SCOPE.md:23; contracts/Treasury.sol:99
REVIEW	Main Review

DESCRIPTION

The deployment specification says Treasury instances have identical bytecode. The `Treasury` constructor initializes OpenZeppelin `EIP712`, which caches the instance address and its domain separator in immutable variables. Solidity embeds these values in deployed runtime code, so instances deployed from the same build with identical constructor arguments have different runtime bytecode and code hashes. The stated identical bytecode guarantee therefore cannot be used as a literal identity check on chain for shard instances.

RECOMMENDATION

State that instances use the same source and compiler artifact while their deployed runtime code contains instance specific immutable values. Compare deployed code with those immutable positions accounted for when verifying shards.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-03** RESOLVED

Cap invariant omits later cap reductions

CATEGORY	Documentation
LOCATION	`docs/REQUIREMENTS.md:98; docs/AUDIT-SCOPE.md:73`
REVIEW	Main Review

DESCRIPTION

The audit invariant says total payouts for an asset and epoch never exceed `dailyCap` and each payout never exceeds `maxPerPayout`, without specifying when those cap values apply. The owner can lower either cap after a successful payout through `setCaps()`, which leaves historical `spent` unchanged. For example, a 10,000-unit payout followed by a reduction of `dailyCap` to 9,999 leaves `spent` at 10,000, above the current cap. That earlier payout can likewise exceed a subsequently reduced `maxPerPayout`.

This is accepted configuration behavior. When historical spend reaches or exceeds the current daily cap, the contract reports zero remaining capacity and rejects further positive payouts. The unqualified written invariant therefore overstates the guarantee that an audit report or integrator can make; the cap checks do not permit an excess payout at execution. The baseline test in `test/Treasury.ts:265-271` exercises a cap reduction below prior spend, zero remaining capacity, and rejection of another payout.

RECOMMENDATION

Qualify both audit invariant lists so each successful payout is bounded by `maxPerPayout` and leaves epoch spend within `dailyCap` under the caps in force when it executes. Explain that later authorized reductions can leave historical epoch spend above the current `dailyCap` or an earlier payout above the current `maxPerPayout`. Keep the existing spend accounting.

PROOF OF CONCEPT https://github.com/GuardianOrg/mirror-creepdius-loan-smartcontracts-cbd15ad22d9cd9a266f1524d7c99640c/blob/92dc8c9e47aaef0f2e36fb1ae63e234a674eeb4e/test/Treasury.ts#L265-L271

RESOLUTION

Status: Resolved.

Guardian: Invariant 3 in both documents still says that lowering either cap blocks further positive payouts until the epoch changes or a cap is raised. Lowering only maxPerPayout still permits a smaller payout in the same epoch when daily capacity remains.

§ 05

Disclaimer.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Guardian to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Guardian's position is that each company and individual are responsible for their own due diligence and continuous security. Guardian's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by Guardian is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis.

Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract's safety and security. Therefore, Guardian does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

§ 06

About Guardian.

Guardian is a smart contract security firm. Every engagement is worked by two independent teams competing on the same scope, each with its own tooling and methods, so no single reviewer's blind spot survives.

$45B+
DIGITAL ASSETS UNDER REVIEW

200+
ENGAGEMENTS COMPLETED

500+
CRITICAL & HIGH FINDINGS REPORTED

Reach us at audits@guardianaudits.com.